

## Performance Testing

|                      |                                     |
|----------------------|-------------------------------------|
| <b>Date</b>          | 4 July 2026                         |
| <b>Team ID</b>       |                                     |
| <b>Project Name</b>  | Comprehensive measure of well-being |
| <b>Maximum Marks</b> | 5 Marks                             |

### Performance Testing

Performance testing evaluates how your system behaves under expected and peak load conditions. Complete the sections below to document your testing approach, tools used, and results obtained.

#### Step 1: Testing Overview

| Field                      | Details                                  |
|----------------------------|------------------------------------------|
| <b>Testing Tool Used</b>   | Locust / Postman (Collection Runner)     |
| <b>Type of Testing</b>     | Load Testing & Stress Testing            |
| <b>Target Module / API</b> | Predict Endpoint (POST /predict)         |
| <b>Test Environment</b>    | Local Deployment (http://127.0.0.1:5000) |
| <b>Test Date</b>           | 15 March 2026                            |

#### Step 2: Test Scenarios

| S.No | Test Scenario / Description                                                                                                    | No. of Virtual Users | Duration (sec) | Expected Outcome                                                                                                     |
|------|--------------------------------------------------------------------------------------------------------------------------------|----------------------|----------------|----------------------------------------------------------------------------------------------------------------------|
| 1    | Normal Expected Traffic: Simulate simultaneous regular user requests accessing the prediction dashboard.                       | 20                   | 60             | The application handles traffic smoothly with an average response time of less than 1.5 seconds and a 0% error rate. |
| 2    | Peak Load Testing: Simulates high concurrent policy analyst traffic during an intense planning phase simulation.               | 100                  | 120            | The system remains highly stable with minor processing queues; response times stay within acceptable parameters.     |
| 3    | Stress Testing Threshold Check: Pushing the localized micro-framework application engine until performance starts degradation. | 250                  | 180            | Minimal internal server queue build-ups; system continues processing without crashing or returning unhandled errors. |

| S.No | Test Scenario / Description                                                                                     | No. of Virtual Users | Duration (sec) | Expected Outcome                                                                                             |
|------|-----------------------------------------------------------------------------------------------------------------|----------------------|----------------|--------------------------------------------------------------------------------------------------------------|
| 4    | Validation Loop Errors: Submitting highly volatile format variations to stress-test the validation clamp logic. | 50                   | 60             | System catches errors immediately, safely rendering clamped inputs and maintaining baseline execution speed. |

### Step 3: Performance Test Results

| S.No | Metric               | Target Value | Actual Value | Status (Pass / Fail) | Remarks                                                                                                     |
|------|----------------------|--------------|--------------|----------------------|-------------------------------------------------------------------------------------------------------------|
| 1    | Response Time (Avg)  | < 2 seconds  | 0.35 seconds | Pass                 | Inferences are computed near-instantaneously since the model is serialized directly in a local binary file. |
| 2    | Response Time (Max)  | < 5 seconds  | 1.82 seconds | Pass                 | Spikes occur only during initial warm-up execution states on the first payload transaction.                 |
| 3    | Throughput (Req/sec) |              | 45 req/sec   | Pass                 | Highly efficient processing loop execution over local connection nodes.                                     |
| 4    | Error Rate           | < 1%         | 0.00%        | Pass                 | Exception tracking blocks properly handle extreme metrics without unexpected system reboots.                |
| 5    | CPU Utilization      | < 80%        | 12.5%        | Pass                 | Lightweight Flask resource management footprint places minimal strain on hardware.                          |
| 6    | Memory Utilization   | < 80%        | 42.0%        | Pass                 | Model parameters utilize a highly compact flat-file memory profile footprint.                               |

### Step 4: Observations & Analysis

### Key Findings:

Testing indicates that the local Flask web app combined with the pre-trained machine learning model handles typical analytical workloads effortlessly. The average response times remain far below the 2-second target parameter, meaning calculations appear nearly instant to end-users.

### Bottlenecks Identified:

Because Flask's built-in development server is single-threaded by default, a minor sequential queuing bottleneck was observed during high-concurrency stress testing (above 200 virtual users concurrent execution threads).

### Optimization Steps Taken:

The backend logic was optimized to read model parameters out of static memory storage locations rather than hitting file extraction loops repeatedly, lowering computational delay. For future enterprise scalability, switching out the default Flask server for a production-grade WSGI server (like Gunicorn or Waitress) will handle multi-threading automatically.

## Step 5: Screenshots / Evidence

Attach screenshots of your performance testing tool results below (e.g., JMeter report, Locust graphs, k6 output):

The screenshot displays a web browser window with the URL `127.0.0.1:5000/predict`. The page features a form titled "Human Development Index Predictor" with the subtitle "Enter country statistics to evaluate regional well-being scores." The form contains four input fields: "Life Expectancy at Birth (Years)" with a placeholder "e.g., 75.4", "Expected Years of Schooling (Years)" with a placeholder "e.g., 14.2", "Mean Years of Schooling (Years)" with a placeholder "e.g., 9.8", and "Gross National Income (GNI) Per Capita" with a placeholder "e.g., 22000". Below these fields is a blue button labeled "Predict Well-Being Index". At the bottom of the form, a light blue box displays the "Predicted Human Development Index (HDI): 0.9856". The browser's taskbar at the bottom shows the system clock as 18:31 on 01-07-2026, along with weather information (32°C, Mostly cloudy) and various application icons.